FraudNet-Synth, Explained Simply

Privacy-Preserving Fraud Detection using Federated Learning and Synthetic Data Augmentation

Shubham Kumar Singh · Palleti Pradeepa · Prachi Yadav · Sathvik D | Guide: Dr. Chethan L S
PES Institute of Technology & Management, Shivamogga (VTU)

The project in one sentence

Four banks work together to train one fraud-catching model **without ever sharing their customers' data**, and the banks that have very few fraud examples create extra **fake-but-realistic** fraud examples to learn from.

1. The problem: fraud is rare

We used a real credit-card dataset with **284,807** transactions. Only **492** are fraud, about **0.17%**. A lazy model that says "everything is genuine" is 99.8% accurate but catches **zero** fraud. So accuracy is misleading, and we judge models by:

- **Precision**: when we say "fraud", how often are we right?
- **Recall**: out of all real frauds, how many did we catch?
- **F1**: one score that balances precision and recall.
- **PR-AUC**: how well the model ranks frauds above genuine transactions overall.

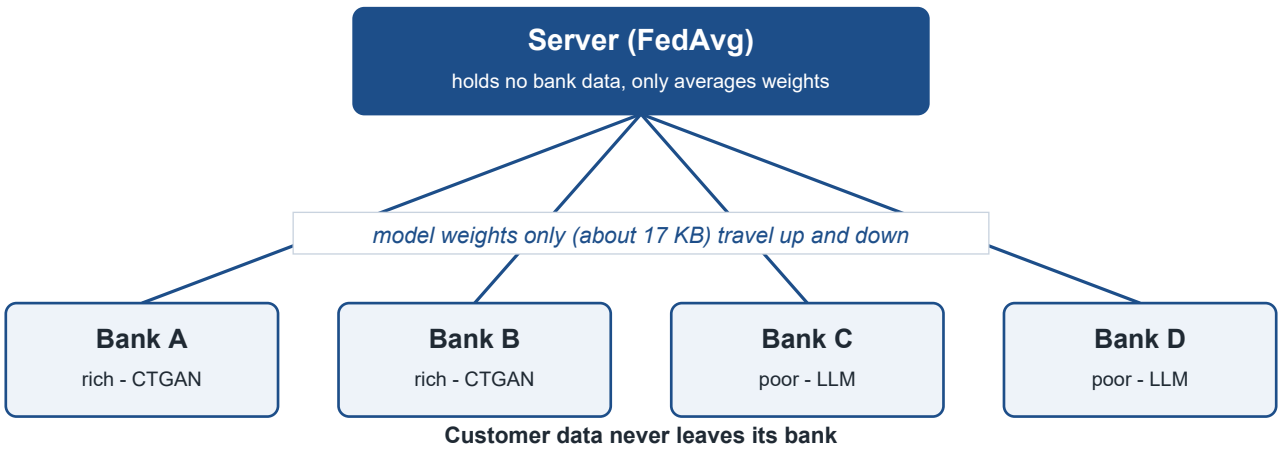
2. Why banks cannot simply share data

Each bank sees only a little fraud. Together they would learn much better, but privacy laws and customer trust stop them from putting customer data in one place.

3. Our solution: federated learning

Analogy: *instead of every student bringing their notebook to one teacher, the teacher sends the* **same question paper** *to four schools. Each school practises at home and sends back only its* **answers summary**. *The teacher combines the summaries into a better paper and sends it out again. That is one* **round**.

- The server sends the model to all four banks **at the same time**, not one after another.
- Each bank trains it on **its own data, on its own computer**.
- Each bank sends back only the learned **weights**, about 17 KB of numbers and never data rows.
- The server **averages** the weights (this is called **FedAvg**). We repeat this for 30 rounds.



4. The banks are unequal (non-IID)

We split the data into four banks on purpose, to be realistic. **Non-IID** simply means the banks' data is different from each other, as it is in real life.

Bank	Type	Fraud rows for training	How it makes extra data
Bank A	Data-rich	105	Augment Mode (CTGAN)
Bank B	Data-rich	82	Augment Mode (CTGAN)
Bank C	Data-poor	28	Schema Mode (LLM)
Bank D	Data-poor	18	Schema Mode (LLM)

5. Making extra fraud examples (synthetic data), two ways

- **Augment Mode, Banks A and B (CTGAN).** Analogy: a forger and a detective. The forger makes fake fraud rows and the detective tries to spot them. Both improve until the fakes look real. It learns only from that bank's own fraud rows.
- **Schema Mode, Banks C and D (an LLM via Groq).** These banks have too few rows for CTGAN, so we ask an AI language model to write rows. For privacy we send it **only averages and ranges, never a real row**.

The method is chosen per bank by how much data it has. This per-bank choice, inside federated learning, is our **research gap**.

6. The quality check (validation gate)

Analogy: a security guard at the door. A fake row is allowed into training only if it:

- looks statistically like real fraud (**fidelity**, checked with SDMetrics);
- is not a copy of a real customer row (**diversity / no memorisation**);
- has the correct columns, types and value ranges (**schema check**, with Pandera).

Example: 814 of Bank B's 1,000 fake rows passed the gate.

7. The fair test: six experiments (arms)

	Real data only	Real + fake data
Each bank alone (lower bound)	Arm 1	Arm 2
Federated (our method)	Arm 3	Arm 4
All data pooled (upper bound, not allowed in real life)	Arm 5	Arm 6

Every arm is tested on the **same hidden test set**, so the comparison is fair. Everything was repeated with 5 different random seeds.

What we found (honest results)

- **Federated learning works:** federated F1 was **0.796**, about as good as pooling all the data, while no data was shared.
- **Poor banks gain the most:** Bank D alone scored only **0.456**; by joining the federation it gets the shared model.
- **Fake data did not reliably help:** in 33 of 36 comparisons the difference was unclear. We report this honestly, because a negative result is still a result.

All numbers come from the project's saved results (docs/RESULTS_ANNEXURE.md, docs/DEMO_SCRIPT.md).

How the software is built (4 layers)

Layer	Technology	Job
Tier 1	React dashboard	What you see: live round charts, six-arm grid, "test a transaction"
Tier 2	Node.js + Express, MongoDB	Login, run management, saving run history
Tier 3	FastAPI + Python ML	The brain: CTGAN, LLM, validation gate, Flower training
Tier 4	Four bank clients	Each holds only its own private data

For the live demo, the four banks run on **four separate laptops** over a phone hotspot. This shows the model really travels between machines while the data stays put.

Honest limitations

- The dataset is public and anonymised, so this is a simulation of banks.
- Reported results come from single-laptop runs. The four-laptop setup is a demo only.
- Extra privacy protections such as differential privacy and secure aggregation are future scope.

60-second script to say out loud

"Fraud is very rare (only 0.17% of transactions), so accuracy is misleading and we measure precision, recall and F1. Banks can't share customer data, so we used federated learning: the server sends a model to four banks, each trains on its own data and sends back only the weights, about 17 KB, and the server averages them. Two banks have many fraud examples and two have very few. The rich banks create extra fake fraud rows with CTGAN; the poor banks use an LLM that sees only averages, never real rows. Every fake row must pass a quality gate before it is used. We compared six setups: alone, federated and pooled, each with and without fake data. Federated reached F1 0.796, close to pooling everything, without sharing any data. Fake data did not give a reliable improvement, and we report that honestly."